Практична робота № 6

Тема: Валідація вхідних даних та обробка запитів у NestJS

Мета роботи

- Опанувати концепцію конвеєра обробки HTTP-запитів у NestJS (`Request Pipeline`) та послідовність виконання його елементів: `Middleware` → `Guards` → `Interceptors` → `Pipes` → `Route Handler` → `Exception Filters`.
- Навчитися реалізовувати декларативну валідацію вхідних тіл запитів (DTO) та параметрів за допомогою бібліотек `class-validator` і `class-transformer`.
- Налаштувати глобальний `ValidationPipe` із захисними механізмами фільтрації корисного навантаження (`whitelist`, `forbidNonWhitelisted`, `transform`).
- Навчитися проєктувати власні пайпи (`custom pipes`) для санітизації та трансформації даних (`PipeTransform`).
- Опанувати створення проміжних обробників (`middleware`) для наскрізного логування HTTP-трафіку та реєстрацію їх через `NestModule`.
- Створити кастомний фільтр виключень (`custom exception filter`) для уніфікації формату відповідей на помилки та інтерцептор (`interceptor`) для моніторингу затримок запитів (`latency`).

Технічне забезпечення

- Операційна система: Windows, macOS або Linux.
- Середовище розробки: Visual Studio Code (із розширенням REST Client або Thunder Client).
- Платформа виконання: Node.js версії 18+ (LTS) та пакетний менеджер npm.
- Проєкт NestJS (створений у межах попередніх практичних робіт або згенерований заново через `@nestjs/cli`).
- Інструменти тестування API: Postman, розширення REST Client (`requests.http`) або консольна утиліта `curl`.
- Термінал / CLI: bash, zsh або PowerShell.
- Система контролю версій: Git.

Короткі теоретичні відомості

1. Архітектура Request Pipeline у NestJS

Кожен вхідний HTTP-запит проходить через суворо визначений ланцюжок компонентів:

```
HTTP Request
      |
      ▼
[ 1. Middleware ] ──────────────────────────► Глобальні або модульні проміжні обробники (логи, CORS)
      |
      ▼
[ 2. Guards ] ──────────────────────────────► Перевірка дозволів та автентифікації (RBAC, JWT)
      |
      ▼
[ 3. Interceptors (Before) ] ──────► Перехоплення запиту до обробника (бенчмаркінг, кеш)
      |
      ▼
[ 4. Pipes ] ──────────────────────────────► Валідація та трансформація вхідних даних (DTO, типи)
      |
      ▼
[ 5. Controller Route Handler ] ──► Виконання бізнес-логіки та звернення до сервісів
      |
      ▼
[ 6. Interceptors (After) ] ───────► Мутація вихідної відповіді (серіалізація, заголовки)
      |
      ▼
[ 7. Exception Filters ] ──────────► Перехоплення невідловлених виключень та генерація JSON-помилок
      |
      ▼
HTTP Response
```

---

2. Декларативна валідація з `class-validator` та глобальний `ValidationPipe`

Бібліотека `class-validator` дозволяє описувати валідаційні правила у вигляді декораторів над полями класів DTO, а `class-transformer` перетворює вхідний сирий JavaScript-об’єкт на екземпляр цього класу.

Приклад DTO звернення до служби підтримки (`CreateSupportTicketDto`):

```
import {
  IsString,
  IsNotEmpty,
  IsEmail,
  IsEnum,
  IsOptional,
  MinLength,
  MaxLength,
} from 'class-validator';

export enum TicketPriority {
  Low = 'LOW',
  Medium = 'MEDIUM',
  High = 'HIGH',
}

export class CreateSupportTicketDto {
  @IsString({ message: 'Заголовок має бути рядком' })
  @IsNotEmpty({ message: 'Заголовок є обов\'язковим' }) ①
  @MinLength(5, { message: 'Мінімальна довжина заголовка — 5 символів' })
  @MaxLength(100)
  readonly title: string;

  @IsEmail({}, { message: 'Некоректний формат email' }) ②
  readonly contactEmail: string;

  @IsEnum(TicketPriority, { message: 'Пріоритет має бути LOW, MEDIUM або HIGH' })
③
  readonly priority: TicketPriority;

  @IsOptional() ④
  @IsString()
  readonly description?: string;
}
```

① Перевірка непорожнього рядка з кастомним текстом помилки.

② Валідація валідного формату адреси електронної пошти.

③ Перевірка відповідності значення допустимим константам перерахування `enum`.

④ Позначення поля як необов’язкового (валідація спрацьовує лише якщо поле передано).

Для активації автоматичної перевірки всього вхідного трафіку в `main.ts` реєструється глобальний `ValidationPipe`:

```
app.useGlobalPipes(
  new ValidationPipe({
    whitelist: true,              // Автоматично видаляє з тіла запиту поля, не
```

```
  описані в DTO ①
4     forbidNonWhitelisted: true,   // Викидає 400 Bad Request, якщо передано зайві
  невідомі поля ②
5     transform: true,              // Автоматично приводить типи (рядки до
  чисел/булевих значень за DTO) ③
6   }),
7 );
```

① Очищення вхідного payload від зайвих полів.

② Блокування спроб передачі недозволених властивостей.

③ Автоматичний кастинг типів значень до зазначених у DTO.

---

3. Створення Custom Pipes для санітизації даних

Пайп — це клас з декоратором `@Injectable()`, що реалізує інтерфейс `PipeTransform`. Він приймає вхідне значення (`value`) та метадані (`metadata`), повертаючи трансформоване значення або викидаючи виключення.

Приклад пайпа очищення пробілів (`TrimStringsPipe`):

```
1 import { PipeTransform, Injectable, ArgumentMetadata } from '@nestjs/common';
2
3 @Injectable()
4 export class TrimStringsPipe implements PipeTransform {
5   transform(value: any, metadata: ArgumentMetadata) {
6     if (typeof value === 'string') {
7       return value.trim(); ①
8     }
9     if (typeof value === 'object' && value !== null) {
10      for (const key of Object.keys(value)) {
11        if (typeof value[key] === 'string') {
12          value[key] = value[key].trim(); ②
13        }
14      }
15    }
16    return value;
17  }
18 }
```

① Безпосередня санітизація, якщо передано окремий рядок.

② Обхід властивостей об’єкта для видалення пробілів на початку й у кінці.

---

4. Middleware логування та реєстрація в `NestModule`

Middleware виконується перед потраплянням запиту в роутер. Має доступ до об’єктів запиту `req`, відповіді `res` та колбеку `next()`.

```
import { Injectable, NestMiddleware, Logger } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class HttpLoggerMiddleware implements NestMiddleware {
  private readonly logger = new Logger('HTTP');

  use(req: Request, res: Response, next: NextFunction) {
    const { method, originalUrl, ip } = req;
    const startTime = Date.now();

    res.on('finish', () => { ①
      const { statusCode } = res;
      const duration = Date.now() - startTime;
      this.logger.log(`${method} ${originalUrl} ${statusCode} - ${duration}ms [${
ip}]`);
    });

    next(); ②
  }
}
```

① Підписка на подію завершення формування HTTP-відповіді.

② Обов’язковий виклик `next()` для передачі запиту наступному обробнику в конвеєрі.

Підключення в кореневому або функціональному модулі:

```
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer.apply(HttpLoggerMiddleware).forRoutes('*');
  }
}
```

---

5. Custom Exception Filter та Interceptors

Фільтр виключень дозволяє перехоплювати помилки та повертати клієнту стандартизований JSON-формат відповіді:

```
import { ExceptionFilter, Catch, ArgumentsHost, HttpException, HttpStatus } from
'@nestjs/common';
```

```
import { Request, Response } from 'express';

@Catch(HttpException) ①
export class GlobalHttpExceptionFilter implements ExceptionFilter {
  catch(exception: HttpException, host: ArgumentsHost) {
    const ctx = host.switchToHttp(); ②
    const response = ctx.getResponse<Response>();
    const request = ctx.getRequest<Request>();
    const status = exception.getStatus();
    const errorResponse = exception.getResponse();

    response.status(status).json({ ③
      statusCode: status,
      timestamp: new Date().toISOString(),
      path: request.url,
      method: request.method,
      errorDetails: typeof errorResponse === 'object' ? errorResponse : { message: errorResponse },
    });
  }
}
```

① Декоратор `@Catch` визначає типи виключень, які перехоплює цей фільтр.

② Перемикання контексту виконання у HTTP-режим.

③ Формування єдиної уніфікованої структури звіту про помилку для клієнта.

Постановка задачі

Для надійної роботи бекенду інтернет-магазину в умовах промислової експлуатації критично важливо забезпечити захист від некоректних даних, автоматичне очищення вхідних параметрів та структурований моніторинг помилок.

У межах практичної роботи необхідно інтегрувати повноцінний **Request Pipeline** до модуля каталогу товарів:

1. Покрити валідаційними декораторами `class-validator` усі контракти DTO відповідно до індивідуального варіанта.
2. Активувати глобальний `ValidationPipe` із жорсткими правилами безпеки (`whitelist: true`, `forbidNonWhitelisted: true`, `transform: true`).
3. Реалізувати власний пайп санітизації тексту (`TrimPipe`).
4. Впровадити `LoggingMiddleware` для фіксації кожного запиту з вимірюванням затримок.
5. Створити глобальний `GlobalHttpExceptionFilter` для формування уніфікованої структури відповідей при помилках.
6. Створити `TimingInterceptor`, що додає службовий HTTP-заголовок `X-Response-Time` до кожної успішної відповіді.

Специфічні правила валідації полів визначаються індивідуальним **варіантом за номером у журналі**:

Таблиця варіантів правил валідації сутності

Варіант	Категорія магазину	Поля та типи для валідації	Специфічні обмеження та валідатори
1	Електроніка та гаджети	`title`, `price`, `warrantyMonths`, `cpu`, `ramGb`	`price > 0`, `warrantyMonths: @Min(1) @Max(60)`, `ramGb: @IsInt() @Min(2)`
2	Книжковий онлайн-магазин	`title`, `price`, `isbn`, `pageCount`, `author`	`isbn: @IsISBN()`, `pageCount: @IsPositive() @IsInt()`, `author: @MinLength(3)`
3	Одяг та взуття	`title`, `price`, `size`, `material`, `color`	`size: @IsEnum(['XS', 'S', 'M', 'L', 'XL'])`, `material: @IsNotEmpty()`
4	Доставка їжі та ресторанів	`title`, `price`, `weightGrams`, `calories`, `isSpicy`	`weightGrams: @Min(10)`, `calories: @Min(0)`, `isSpicy: @IsBoolean()`
5	Онлайн-аптека	`title`, `price`, `dosageMg`, `isPrescriptionRequired`, `expirationDate`	`dosageMg: @IsPositive()`, `isPrescriptionRequired: @IsBoolean()`, `expirationDate: @IsDateString()`
6	Будівельні матеріали	`title`, `price`, `unitOfMeasure`, `weightPerUnitKg`, `isHeavyCargo`	`unitOfMeasure: @IsEnum(['kg', 'm2', 'm3'])`, `weightPerUnitKg: @IsPositive()`
7	Автозапчастини	`title`, `price`, `oemCode`, `condition`, `carBrand`	`oemCode: @Length(6, 20)`, `condition: @IsEnum(['NEW', 'REFURBISHED'])`
8	Квитки на події	`title`, `price`, `venue`, `sector`, `seatNumber`	`sector: @IsInt() @Min(1)`, `seatNumber: @IsInt() @Min(1)`, `price: @Min(50)`
9	Онлайн-курси та підписки	`title`, `price`, `durationHours`, `difficultyLevel`, `instructor`	`durationHours: @IsPositive()`, `difficultyLevel: @IsEnum(['BEGINNER', 'MIDDLE', 'ADVANCED'])`
10	Зоотовари	`title`, `price`, `petCategory`, `packageWeightKg`, `brand`	`petCategory: @IsEnum(['CAT', 'DOG', 'BIRD', 'FISH'])`, `packageWeightKg: @Min(0.1)`

> Якщо номер у журналі перевищує 10, варіант обчислюється за формулою: `((номер - 1) % 10) + 1`.

Завдання

Практична робота складається з трьох послідовних рівнів. Кожен наступний рівень базується на попередньому та розширює функціональність розроблюваного рішення.

Рівень 1. Базовий: підключення `class-validator` та валідація DTO

1. **Встановлення валідаційного пакету:**
   - Встановити необхідні залежності у проєкт NestJS:

     ```
     npm i class-validator class-transformer
     ```

2. **Налаштування глобального `ValidationPipe`:**
   - У файлі `src/main.ts` налаштувати глобальну валідацію:

     ```
     app.useGlobalPipes(
       new ValidationPipe({
         whitelist: true,
         forbidNonWhitelisted: true,
         transform: true,
       }),
     );
     ```

3. **Оновлення та валідація контрактів DTO:**
   - Відкрити `Create<Resource>Dto` свого варіанта та розмітити кожну властивість валідаційними декораторами:
     - `@IsString()`, `@IsNotEmpty()` для текстових полів;
     - `@IsNumber()`, `@IsPositive()` для вартості (`price`);
     - декоратори специфічних обмежень згідно зі своїм **варіантом** (енуми, діапазони, цілочисельність);
   - Відкрити `Update<Resource>Dto`:
     - Додати до всіх властивостей декоратор `@IsOptional()`, зберігши правила перевірки типів.
4. **Базова перевірка валідації:**
   - Надіслати POST-запит із некоректними типами даних (наприклад, рядок замість числа або від'ємна ціна) та зафіксувати отримання автоматичної відповіді `400 Bad Request` із масивом детальних описів порушених правил.

Рівень 2: санітизація даних через Custom Pipe та мережеве Middleware

1. **Створення Custom Pipe (`TrimPipe`):**
   - Створити каталог `src/common/pipes/` та файл `trim.pipe.ts`.
   - Реалізувати клас `TrimPipe implements PipeTransform`:
     - Якщо вхідне значення є рядком — застосувати метод `.trim()`;
     - Якщо передано об'єкт — рекурсивно обійти всі рядкові поля та видалити зайві пробіли на початку й у кінці;
     - Повернути очищене значення.
   - Підключити `TrimPipe` глобально або для окремого обробника створення товару.
2. **Створення та реєстрація `HttpLoggingMiddleware`:**
   - Створити каталог `src/common/middleware/` та файл `http-logger.middleware.ts`.
   - Реалізувати `HttpLoggingMiddleware implements NestMiddleware`:
     - Зчитувати метод, URL, IP-адресу клієнта та засікати початковий час запиту;
     - Через підписку на подію завершення відповіді `res.on('finish', …)` вираховувати тривалість обробки у мілісекундах;
     - Логувати структуроване повідомлення через стандартний клас `Logger` фреймворку NestJS: `[HTTP] <METHOD> <URL> <STATUS_CODE> +<LATENCY>ms`.
   - Зареєструвати middleware у `AppModule`, реалізувавши інтерфейс `NestModule` та метод `configure(consumer: MiddlewareConsumer)` для всіх маршрутів (`forRoutes('*')`).

Рівень 3. Просунутий: Custom Exception Filter, Interceptor та наскрізне тестування

1. **Реалізація Custom Exception Filter (`GlobalExceptionFilter`):**
   - Створити каталог `src/common/filters/` та файл `http-exception.filter.ts`.
   - Реалізувати фільтр із декоратором `@Catch(HttpException)`:
     - Отримувати доступ до об'єктів `Request` та `Response` через `host.switchToHttp()`;
     - Формувати єдину уніфіковану JSON-відповідь клієнту:

```
{
  "success": false,
  "statusCode": 400,
  "timestamp": "2026-09-09T22:40:00.000Z",
  "path": "/gadgets",
  "method": "POST",
  "error": "Bad Request",
```

```
  "message": [
    "price must be a positive number",
    "warrantyMonths must not be less than 1"
  ]
}
```

- Зареєструвати фільтр глобально у `main.ts`: `app.useGlobalFilters(new GlobalExceptionFilter());`.

2. **Реалізація Interceptor моніторингу швидкодії (`TimingInterceptor`):**
   - Створити каталог `src/common/interceptors/` та файл `timing.interceptor.ts`.
   - Реалізувати клас `TimingInterceptor implements NestInterceptor`:
     - Фіксувати час надходження запиту;
     - За допомогою RxJS оператора `tap()` перехоплювати завершення відповіді;
     - Встановлювати клієнтський HTTP-заголовок `X-Response-Time: <Time>ms` через `response.setHeader()`;
     - Логувати час роботи роут-хендлера в консоль.
   - Підключити інтерцептор глобально через `app.useGlobalInterceptors(new TimingInterceptor());`.
3. **Комплексне тестування конвеєра (`requests.http`):**
   - Створити набір перевірочних запитів у файлі `requests.http`:
     - **Тест 1 (Валідний):** створення сутності з валідними даними та пробілами в назві (перевірка очищення через `TrimPipe`, статусу `201 Created` та наявності заголовка `X-Response-Time`);
     - **Тест 2 (Фільтрація заборонених полів):** надсилання невідомого поля (наприклад, `"hackerField": "malicious"`) — перевірка спрацьовування `forbidNonWhitelisted` та генерації помилки `property hackerField should not exist`;
     - **Тест 3 (Некоректні типи даних):** передача рядка в чисельне поле — перевірка роботи `GlobalExceptionFilter` та уніфікованої JSON-структури;
     - **Тест 4 (Query-трансформація):** передача `?limit=5` у GET-запит — перевірка автоматичного приведення рядка до числа через опцію `transform: true`.
   - Додати скриншоти консолі сервера з логами `HttpLoggingMiddleware` до звіту `README.md`.

Приклад очікуваного виведення програми

```
[Nest] 21340  - 09/09/2026, 22:45:00     LOG [NestFactory] Starting Nest
application...
[Nest] 21340  - 09/09/2026, 22:45:00     LOG [InstanceLoader] AppModule dependencies
```

```
initialized +10ms
[Nest] 21340  - 09/09/2026, 22:45:00     LOG [RoutesResolver] GadgetsController
{/gadgets}: +4ms
[Nest] 21340  - 09/09/2026, 22:45:00     LOG [RouterExplorer] Mapped {/gadgets, POST}
route +2ms
[Nest] 21340  - 09/09/2026, 22:45:00     LOG [NestApplication] Nest application
successfully started on port 3000 +2ms

[HTTP] POST /gadgets 400 - 4ms [::1]
[TIMING] Маршрут POST /gadgets виконано за 3.82ms

=== ПЕРЕВІРКА ВІДХИЛЕННЯ ЗАПИТУ З ЗАБОРОНЕНИМИ ПОЛЯМИ ===

> POST /gadgets HTTP/1.1
> Host: localhost:3000
> Content-Type: application/json
{
  "title": "  Ноутбук Pro 16  ",
  "price": -500,
  "unknownPayload": "hack"
}

< HTTP/1.1 400 Bad Request
< Content-Type: application/json; charset=utf-8
< X-Response-Time: 4.12ms
{
  "success": false,
  "statusCode": 400,
  "timestamp": "2026-09-09T22:45:10.120Z",
  "path": "/gadgets",
  "method": "POST",
  "error": "Bad Request",
  "message": [
    "property unknownPayload should not exist",
    "price must be a positive number"
  ]
}
```

Контрольні питання

1. Опишіть черговість проходження запиту крізь конвеєр `NestJS Request Pipeline` (від отримання мережевого сокету до генерації фінальної відповіді).
2. Яку роль відіграють опції `whitelist` та `forbidNonWhitelisted` у конфігурації глобального `ValidationPipe`? Від якого вектора кібератак вони захищають сервер?
3. Що робить опція `transform: true` у `ValidationPipe`? Чому без неї значення з `@Query()` та `@Param()` завжди залишаються рядками?

4. У чому полягає відмінність між `Middleware` та `Interceptor` у NestJS? У яких сценаріях слід обрати кожен із них?
5. Для чого призначений інтерфейс `PipeTransform`? Які аргументи приймає метод `transform(value, metadata)`?
6. Як працює декоратор `@Catch(HttpException)` у фільтрі виключень? Як за допомогою `host.switchToHttp()` отримати доступ до платформних об'єктів Express?
7. Як створити зв'язку між `ValidationPipe` та бібліотекою `class-transformer` для коректної валідації вкладених об'єктів за допомогою `@ValidateNested()`?
8. Яким чином оператор RxJS `tap()` в інтерцепторі дозволяє виконати дію над вихідним потоком відповіді, не змінюючи саме тіло даних?

Критерії оцінювання

Оцінювання здійснюється за шкалою повноти реалізації рівнів практичної роботи:

- **«Задовільно» (Рівень 1):** реалізовано базовий прототип, створено структуру проєкту, програма успішно компілюється та виконує базові сценарії.
- **«Добре» (Рівень 2):** виконано базовий рівень та реалізовано ключову функціональність заняття, забезпечено надійність і безпеку виконання коду.
- **«Відмінно» (Рівень 3):** повністю завершено всі три рівні завдання з дотриманням кращих інженерних практик, чистоти архітектури та оптимізації.
- **Усний захист:** обов'язкове пояснення логіки коду та впевнені відповіді на контрольні питання викладача.

Вимоги до звітності

1. Опублікувати вихідний код у Git-репозиторії.
2. Оформити файл `README.md` з описом проєкту, інструкцією запуску та обов'язковими скриншотами роботи програми (включаючи демонстрацію відсутності помилок).
3. Продемонструвати роботу програми викладачеві та захистити результати.